Expt 01
23.09.2026

1. BASIC NETWORKING COMMANDS.

AIM:

Study of various Network commands used in Linux and Windows:

BASIC NETWORKING COMMANDS:

① arp-a:-

ARP is short form address resolution protocol, it will show the IP address of your computer along with the IP address of and MAC address of your router.

output:-

Address	Hw Address	Hw Type	Flags Mask
(172.16.10.116)	at dg:bb:d:c5:c8:c6	ether	C
172.16.8.188	7c:57:58:58:45:36	ether	C
172.16.11.74	00:27:0e:13:ef:04	ether	C
172.16.10.110	d8:bb:d:c5:ca:0a	ether	C
172.16.8.150	00:27:00:13:cd:88	ether	C

② hostname:

This is the simplest of all TCP/IP commands. It simply displays the name of your computer

output:-

DESKTOP-F4M5ER.

③ ipconfig/all:

Displays complete TCP/IP network configuration, including IP address, subnet mask, gateway, DNS, DHCP and mac address

op: window IP configuration.

Host name . . . . . : Gayathri

Primary DNS Suffix . . . . . :

Node type . . . . . : Mixed

IP routing Enabled . . . . : NO

WINS proxy enabled . . . : NO

4. nbtstat -a :
displays the NET BIOS name table of remote computer to help troubleshoot Net Bios name resolution.

o/p: Displays protocol statistics and current TCP/IP connections using NBT

NBTSTAT [[-a Remote Name] [-A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]

-a (adapter status) lists the remote machine's name table given its name.

-A (Adapter status) list the remote machine's name table given its IP address.

-n (names) list local Net Bios names.

5. netstat :
Displays active network connections, listening, ports and network statistics.

o/p : Active connections

Proto	Local address	Foreign Address	state
TCP	[::1]:1521	GAYATHRI:49672	Established
TCP	[::1]:49672	GAYATHRI:49672	Established

6. netstat -r :-
Displays the routing table of the computer.

o/p: Interface list

21 ... 90 ff 26 da 62 66 ... TAP Window Adapter V9

6 ... 6e 3d 3d 7f 5d 30 ... Realtek RTL 8851BE WiFi 6 802.11ax PCIe Adapter #3

3 ... 3a 8d 3d 7f 5d 30 ... Realtek RTL 8851BE WiFi 6 802.11ax PCIE Adapter #5

2 ... 38 3d 3d 7f 5d 30 ... Realtek RTL 8851BE WiFi 6 802.11ax PCIe Adapter

1 ..... Software loopback Interface

7. nslookup :
Finds the IP address of a domain name using the DNS server.

o/p :
default server : unknown
Address : 127.0.0.1

8. pathping :
Traces the route to a destination by measures packet loss at each network hop.

o/p:
usage : pathping [-g host-list] [-h maximum-hops] [-i address] [-n] [-p period] [-q num-queries] [-w timeout] [-4] [-6] target name.

9. ping :
Test connectivity b/w your computer by another computer or website using ICMP

o/p:

ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS] [-r count] [-s count] [[-j host-list] [-k host-list]] [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p] [-4] [-6] target-name

10. route:
Displays the IP address table used by the operating system to route network traffic.

O/P:
Route [-f] [-p] [-4|-6] command [destination] [mask netmask] [gateway] [metric metric] [if interface]

IMPORTANT LINUX NETWORKING COMMANDS:

1. ip <option> <object> <command>:
The ip command is one of the basic commands. The commands can show address information, manipulate routing.
eg: ip address show

O/P:
lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state unknown group default qlen 1000
inet 127.0.0.1

2. ifconfig:
The ifconfig command was/is a staple in many sysadmin's tool belt for configuring and troubleshooting networks. It has since been replaced by the ip command discussed.

O/P: Windows IP Configuration
Unknown adapter MC after VPN:
Media state ..... : Media disconnected.
Connection specific DNS Suffix.

3. Mtr: MTR is a program with a command-line interface that serves as a network diagnostic and troubleshooting tool. This command combines the functionality of the ping and traceroute commands.

eg: mtr google.com.

Host	Loss %	Snt	Last	Avg	Best
1. 172.16.8.1	0.0%	134	0.1	0.2	0.1
2. 172.97.163.129	2.6%	134	3.9	5.2	1.1
3. ???	2.2%	134	46.6	18.6	4.1

4. tip dump: commands is designed for capturing and displaying commands
eg: dnf install -y tcp dump
O/P: tcp_dump -D

~~O/P~~:

5. Ping: - ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending internet control message protocol (ICMP) Echo request messages.

eg: ping google.com [142.250.67.46] with 32 bytes of data:
Reply from 142.250.67.46: bytes=32 time=9ms TTL=119
Reply from 142.250.67.46: bytes=32 time=3ms TTL=119
Reply from 142.250.67.46: bytes=32 time=3ms TTL=119
Reply from 142.250.67.46: bytes=32 time=3ms TTL=119

Ping statistics for 142.250.67.41:
packets: Sent=4, Received=4, Lost=0 (0% Loss)

Approximate round trip times in milli-seconds: minimum = 3ms, maximum = 4ms, Average = 4ms

Result:

Basic networking commands and Linux commands are learned and executed.

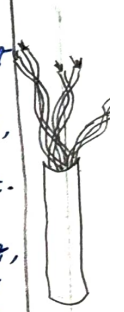
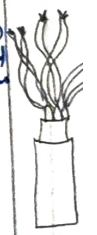
Practical-2

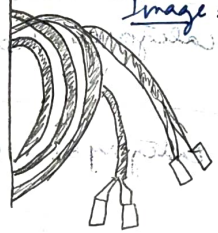
AIM: Study of different types of network cables.

a. understand different types of network cable.

Different type of cables used in networking are:

1. Unshielded Twisted pair (UTP) cable.
2. Shielded Twisted pair (STP) cable.
3. Coaxial cable
4. Fibre optic cable.

Cable type	Category	Maximum Data Transmission	Advantages / disadvantages	Application / Use	Image
UTP	Category 3	10 Mbps	Advantages: · Cheaper in cost · easy to install as they have a smaller overall diameter. Disadvantages: · More prone to (EMI) Electromagnetic interference and noise.	10 Base - T ethernet Fast ethernet, Gigabit ethernet. Fast ethernet, Gigabit Ethernet.	
	Category 5	up to 100 Mbps			
	Category 5e	1 Gbps			
STP	Category 6/6a	10 Gbps	Advantages: · Shielded · Faster than UTP. · Less susceptible to noise and interference. Disadvantages · Expensive · Greater installation effort	Gigabit Ethernet LSI) widely used in data centers Gigabit Ethernet, 10G Ethernet (10GBASE-T).	
SSTP.	Category 7	10 Gbps			

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Adv: • High bandwidth. • Immune to interference • Low loss bandwidth. • Versatile DisAdv:- • Limited distance • Cost • Size is bulky	• Speed of signal is 500 m Television network high speed internet connections Image: 
fibre optics cable	Single mode multimode	100 Gbps	Adv:- • High speed • High bandwidth • High security • Long distance DisAdv. • Expensive • Requires skilled installers	• Maximum distance of fibre optics cable is around 100 meters Image: 

Result:-

Thus different types of cables are studied successfully.

Practical-3:

AIM: To study the packet tracer tool installation and user Interface overview

Analyze the behaviour of network devices using CISCO PACKET TRACER simultaneously.

1. From the network component box, click and drag and drop the below components:
   a. 4 Generic PCs and One Hub.
   b. 4 Generic PCs and One Switch.

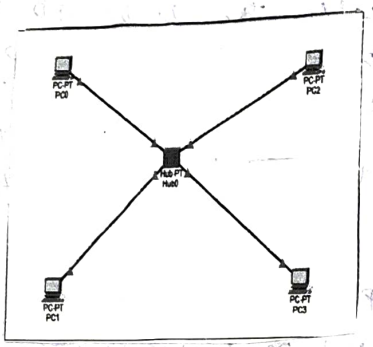
2. Click on connections:
   a. Click on copper Straight-Through cable.
   b. Select one of the PCs and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3 PCs to the HUB.
   c. Similarly connect 4 PCs to the switch using copper straight-through cable.

3. Click on the PCs connected to hub, go to the Desktop tab, click on ip configuration, and enter an IP address and subnet mask. Here, the default gateway and DNS server information is not needed as there are only two end devices in the network.
   Click on the PDU (message icon) from the common tool bar.
   a. Drag and drop it one of PC (source machine) and then drop it one another PC.

4. Observe the flow of PDU from source PC to destination PC by selecting the realtime mode of simulation.

5. Repeat step #3 to step #5 for the PCs connected to the switch.

6. Observe how HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviour of switch and HUB.



Result:

CISCO PACKET TRACER are learned & executed successfully.

21/7/26

PRACTICAL - 4

SETTING UP A LOCAL AREA NETWORK (LAN)

AIM:

To design, set up and configure a local Area Network (LAN) connecting four computers through a network switch, assign static IP addresses to all devices, verify connectivity using the ping command, and demonstrate resources / data sharing across the network.

EQUIPMENT REQUIRED:

S.NO	EQUIPMENT	QUANTITY	SPECIFICATION
1.	PERSONAL COMPUTERS (PC)	4	Ethernet (NIC) enabled
2.	NETWORK SWITCH	1	8/16/24 port. unmanaged / managed
3.	ETHERNET CABLES (Cat5e / Cat6)	4	Straight through. RJ-45 connectors.

THEORY:

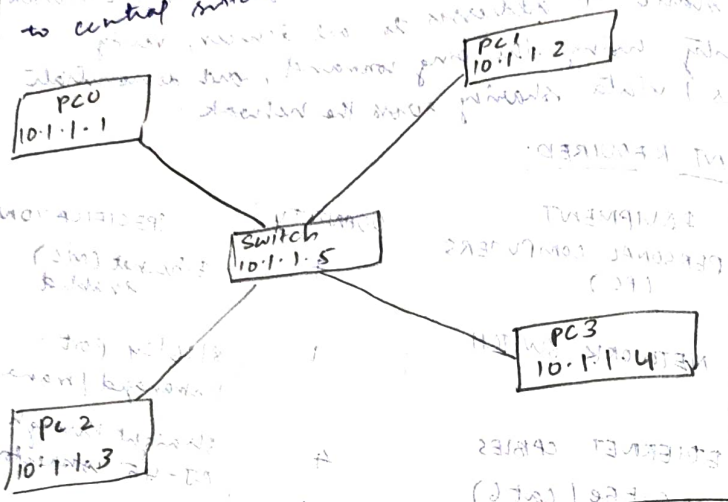
A Local Area Network (LAN) is a network that connects computers and devices within a limited geographical area such as a home, office (or) laboratory allowing them to share resources such as files, printers, and internet connection.

A star topology is used in this experiment, where all end devices (PCs) are individually connected to a central network switch. The switch operates at Layer 2 of the OSI model and forwards data frames between connected devices based on their MAC addresses.

Every device on a LAN must be assigned an IP address within the same network range (as determined by the subnet mask)

Network Diagram:-

The LAN was implemented using a star topology as shown below, with 4 PCs connected individually to central switch.



IP configuration Table:

Device	IP Address	Subnet mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0
Switch (management)	10.1.1.5	255.0.0.0	

Implementation in Cisco Tracer:

Since physical hardware was limited, the LAN was also implemented and verified using Cisco Packet Tracer Simulation software, following the steps below

Step 1: Place the Devices

1. Open Cisco Packet Tracer
2. From the device category panel, select End devices and drag 4 PCs onto the workspace
3. Select switches and drag 1 switch (eg: 2960 series) onto the workspace

Step 2: cable the Devices

4. Click connections → select copper straight through cable
5. Click each PC, then click the switch and select an available Fast Ethernet port to complete each connection
6. Repeat for all four PCs (port Fa0/1 to Fa0/4). Green link lights confirm a successful ~~physical~~ connection.

Step 3:- Assign IP Addresses to PCs

7. Click a PC → desktop tab → IP configuration
8. Enter the static IP addresses, & Subnet mask as per the IP table.
9. Click switch → CLI Tabel
10. Enter the following commands to assign a management IP.

IP addresses 10.1.1.5 2

Step 5: Verify connectivity using ping

11. using a pc → desktop tab → command prompt.

12. using the ping command to test connectivity to other pcs ping 10.1.1.2

13. Successfully "Reply from ...." messages confirm connectivity

14. Add a server device (end device → server) pc) by connect it to switch.

15. Assign an IP addresses (10.1.1.10) Subnet mask in its desktop → IP config tab

16. Open the servers services tab by end the HTTP services

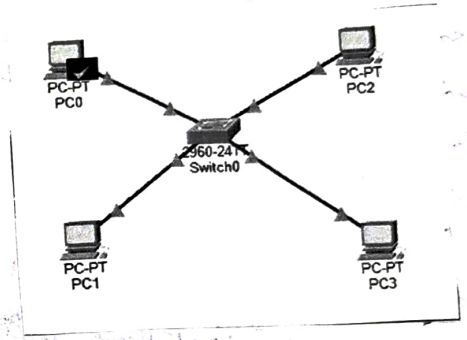
17. From another pc. Open the web by enter the number IP addresses to load page.

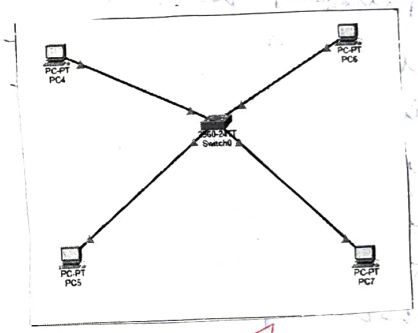
18. Switch to simulation mode.

19. Send a ping (or) web req b/w 2 devices. Use play (capture controls to step through the packet as it travel hop by hop through switch.

Observation:

All 4 pcs were successfully connected to the switch in a star topology by unique static IP addresses within the same subnet. Pings tests b/w all devices are returned successful replies & confirming end to end connectivity





Result:

Thus LAN is successfully studied.

Exp 5: Experiments on packet capture tool - Wireshark

AIM:
Experiments on packet capture tool - Wireshark packet sniffer.
- sniffs messages being sent / received from/by your computer
- stores & display the contents of the various protocols fields in msg.
- passive program:
packet sniffer stimulus diagnostic tool
- tcpdump.
- Wireshark.

PROCEDURE:
capturing packets:-
After downloading & installing wireshark, launch it & double-click the name of a network interface under capture to start capturing packets on that interface & as soon as you click the interfaces name, you'll see the packets starts to appear in real time. Wireshark captures.

If you have promiscous mode enabled - its enabled by default you'll also see all the other packets on the network instead of only packets addressed to your network. To check if promiscous mode is enabled;

click capture > options & verify the "enable promiscous mode on all interfaces"

Create a filter to display only TCP packets & inspect the packets by pressing the flow graph.

- select Local Area Network in wireshark.
- go to capture option.
- Select stop capture automatically 100.
- Then click start capture
- Search tcp packets in search bar.
- To see flow graph click statistics flow graph.

Save the packets

create a filter to display only ~~or~~ ARP packets by inspect the packets.

RESULT:

Thus ~~experiments~~ on packet capture tool wireshark is studied successfully.

Ex: 6 HAMMING CODE

Aim:

Write a program to implement errors detection and correction using hamming code concept. Make a test run to project data stream and name error correction function.

Error correction at data link layer:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from sender to receiver

Code:

```
def sender():
    data = input("Enter text: ")
    # convert text to binary
    binary = ""
    for ch in data:
        binary += format(ord(ch), '08b')
    print("Binary data:", binary)
    # find number of parity bits
    m = len(binary)
    r = 0
    while 2**r < m + r + 1:
        r += 1
    n = m + r
    code = ['0'] * (n+1)
    # put data bits
    j = 1
    for i in (i-1):
```

```
code [
    j+1 = 1
# calculate parity bits.
for p in range (r):
pos = 2** p.
parity = 0
for pi in range (1, n+1):
    if i & pos:
        parity ^= int (code [i])
code[pos] = str (parity)
hamming = "". join (code[1:]).
print ("Hamming code:", hamming)
# save to channel
with open ("channel", "w") as f:
    f. write (hamming).
```

Sender :

```
# r1, r2, r4 and r8 are redundancy bits
text=input ("Enter the text")
# convert text to binary
binary = ""
for ch in text:
    binary += format (ord (ch), '08b')
print ("Binary data:" binary)
channel = ""
# process 8 data bits at a time
for i in range (0, len (binary), 8):
    d = binary [i : i+8].
```

```
# 12 bit Hamming code.
# Position: 1 2 3 4 5 6 7 8 9 10 11 12.
#          r1 r2 d1 r4 d2 d3 d4 d5 d6 d7 d8
code = [0] * 13
# put data bits
code [3] = d[0]
code [5] = d[1]
code [6] = d[2]
code [7] = d[3]
code [10] = d[4]
code [11] = d[5]
code [12] = [d7]
# calculate r1
    code [1] = str (
        int (code [3]) ^ int (code [5]) ^ int
                                    (code [7]) ^
    ( int (code [9]) ^ int ( code [11])
    )
# calculate r2
    code [2] = str (
        int (code [3]) ^ int ( code [6]) ^ (int
    (code [7]) ^ int code [0] ^ int (code [11])
    ).
# calculate r4
        code [4] = str (
        int (code [5]) ^ int (code [6]) ^ int (code
                                    [7]) ^
        int (code [12])
        )
# calculate r8
    code [8] = str (
        int (code [9] ^ int (code [10]) ^
    int (code [11] ^ int (code [12]))
```

```
# Add 12 bits to channel
for i in range(1,13):
    channel += code[i]

print("Hamming code:", channel)
# Save hamming code in channel.txt
with open("channel.txt","w") as f:
    f.write(channel)
print("Hamming code saved in channel.txt")
```

Receiver:

```
# r1, r2, r4 and r8 are redundancy bits
# Read data from channel
with open("channel.txt","r") as f:
    received = f.read()
print("Received data:", received)
corrected = ""
# process 12 bits at a time
for x in range(0, len(received), 12):
    c = ['0'] + list(received[x:x+12])
    # check r1
    r1 = (
        int(c[1]) ^ int(c[3]) ^ int(c[5]) ^ int(c[7])
        ^ int(c[9]) ^ int(c[11])
    )
# check r2
    r2 = (int(c[2]) ^ int(c[3]) ^ int(c[6]) ^ int(c[7])
    ^ int(c[10]) ^ int(c[11])
    )
# r4
    r4 = (
        int(c[4]) ^ int(c[5]) ^ int(c[6]) ^ int(c[7])
        ^ int(c[12])
    )
    # check r8
    r8 = (
        int(c[8]) ^ int(c[9]) ^ int(c[10]) ^ int(c[11])
        ^ int(c[12])
        )
    # Find error position
    error = r1*1 + r2*2 + r4*4 + r8*8
    if error == 0:
        print("No error detected")
    else:
        print("Error detected at position:", error)
        # correct the error
        if c[error] == '0':
            c[error] = '1'
        else:
            c[error] = '0'
        print("Error corrected")
# Removing redundancy bits
    corrected += (
        c[3] + c[5] + c[6] + c[7] + c[9] + c[10] +
        c[11] + c[12]
    )
```

```
output = ""
for n in range (0, len (corrected), 8):
    byte = corrected [n : n+8]
    if len (byte) == 8
        output += chr (int (byte, 2))

print ("original text", output)
```

output:

```
Enter the text: hello
binary data: 011010000110010001101100011
    011000110111
Hamming code: 010011011000100111000
    000011001100000011001100110110001111
Hamming code sent in channel: 
received data = 010001011000100111000010100
    001100000011001100110111001111
Error detected at position: 5
Error corrected
```

Exp: 07:

Sliding window protocol

Aim:

write a program to implement flow chart of data link layer using sliding window protocol simulate the flow of frames from one node to another.

code:

Sender:

```
import time
import os
window_size = int (input ("Enter window size: "))
message = input ("Enter the message: ")
frames = []
for i, ch in enumerate (message, start=1):
    frame.append (i, ch)
Sender_buffer = "sender_buffer.txt"
receiver_buffer = "receiver_buffer.txt"
start = 0
while start < len (frames):
    window = frames [start : start + window_size]
    print ("\n Sender")
    with open (sender_buffer, "w") as file:
        for frame_no, data in window:
            file.write (f"{frame_no}, {data}\n")
            print ("Frame", frame_no, ":", data)
print ("Frames written to sender_buffer.txt")
input ("\n Press enter to continue:")
if not os.path.exists (receiver_buffer):
    print ("receiver_buffer.txt not found")
```

```
        continue
    with open(receiver_buffer,"r") as file:
        response = file.read().strip()
    print("Received:",response)
    if response.startswith("ACK:"):
        ack_no = int(response.split(":")[1])
        expected_ack = window[0][0]
        if ack_no == expected_ack:
            print("Ack", ack_no, "received")
            print("Window moved")
            start = start + window_size
    else:
        print("Wrong Ack")
        print("Expected:", expected_ack)
        print("Received", ack_no)
        print("Resending frames")

elif response.startswith("NACK:"):
    nack_no = int(response.split(":")[1])
    print("NACK", nack_no, "received")
    print("Resending from frames", nack_no)
    start = nack_no - 1

else:
    print("Invalid response")

time.sleep(1)
print("\n All frames are sent")
```

```
receiver:
    import os
    sender_buffer = "sender_buffer.txt"
    received_buffer = "Received_Buffer.txt"
    expected_frame = int(input("Enter expected
frame no: "))
    print("\n receiver")
    if not os.path.exists(sender_buffer):
        print("sender buffer not not forced")

    else:
        with open(sender_buffer,"r") as file:
            lines = file.readlines()
        error = False
        last_frame = expected_frame-1
        print("Frames - Received")
        for line in lines:
            line = line.strip()
            if line == "":
                continue
            parts = line.split(",")
            frame_no = int(parts[0])
            data = parts[1]
            print("Frame" frame_no:", data)
            if frame_no == expected_frame:
                print("Frame", frame_no,
                      "received correctly")
                last_frame = frame_no
                expected_frame += 1
            else:
                print("Error in frames", frame_no
                print("Expected frames:", expected_fra
                error = True
        with open(receiver_buffer,"w") as
            file.write("NACK:{expected_frame
        print("NACK", expected_frame, "sent"
```

break

it not exp

with open (receiver_buffer, "w+") as fh:
    fh.write (f"ACK: {last_frame}")
print ("Ack", last_frame, "sent")

output:

Sender (window 1):
Enter window size: 3
Enter the message: HELLO
sender:
Frame 1: H
Frame 2: E
Frame 3: L
Frames written to sender_buffer.txt
Received: ACK: 3
ACK 3 received.
Windows moved.
Sender
Frame 4: O
Frame 5: 0
Frame written to sender_buffer.txt
received: ACK: 5
Ack 5 received
window moved
All frames sent successfully.

Receiver (window 2:):
Expected frame no: 1
Receiver
Frame 1: H
Received correctly
Framed: E

received correctly
Frame 3: #L
Received correctly
ACK 3 sent
Expected frame no: 4
Receiver
Frame 4:
received correctly
Frame 5:
received correctly.

RESULT:

Thus, the frames are sent and received using the sliding window protocol.

Exp: 08

(a) NMAP TO DISCOVER

SCANS ON THE TRY HACKME PLA

Aim: To discover live hosts using NMAP scans ICMP, TCP/UDP on the Try hack me platform.

Introduction:

When targetting a network we need an efficient tool to handle repetitive tasks. This tool should help us find out which systems are active and what services are running on those systems.

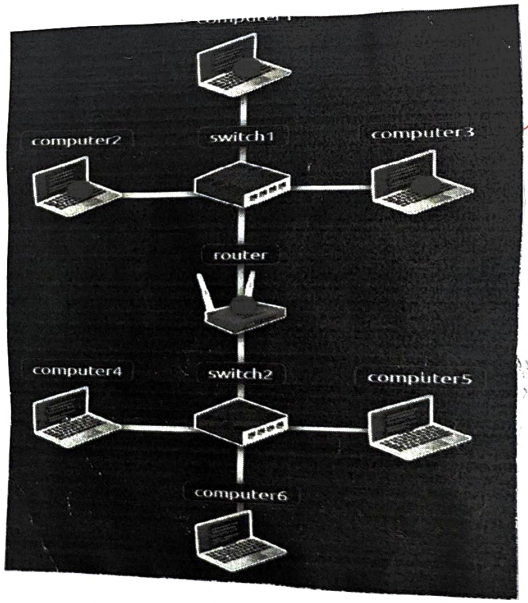
This room is the first of four in this Nmap series. These four rooms are also part of the network security module:

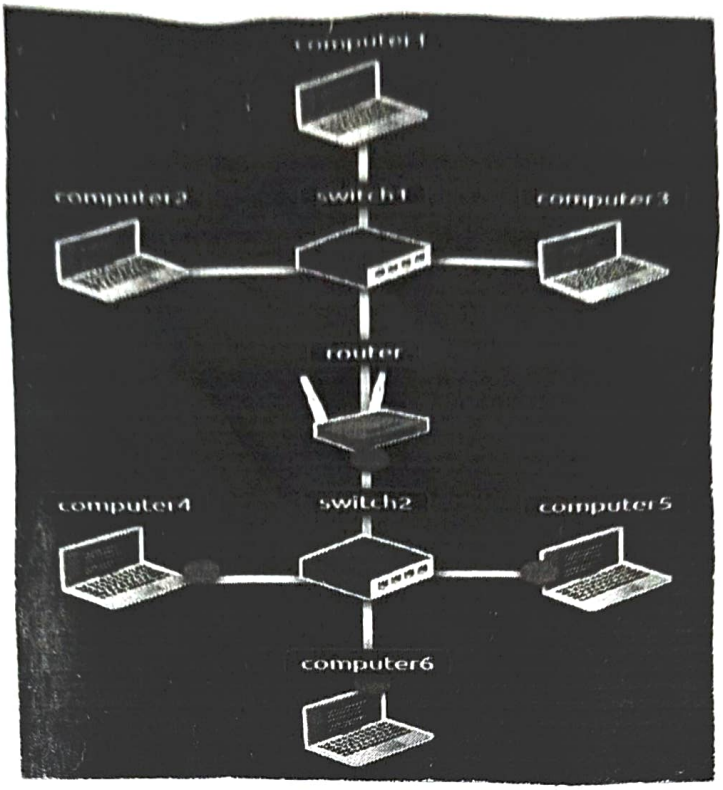
NMAP live Host Discovery

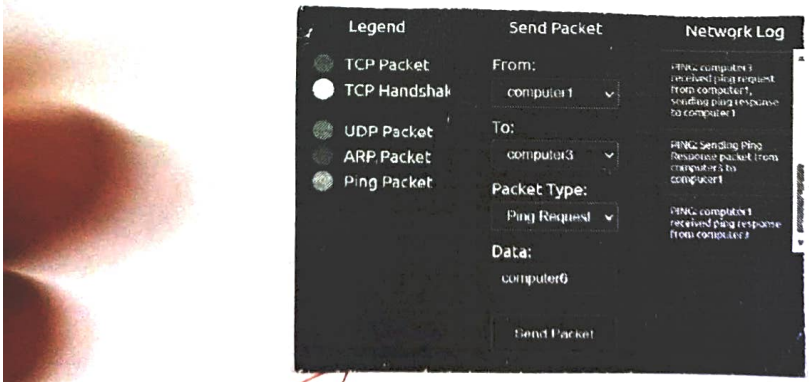
NMAP Basic port scans

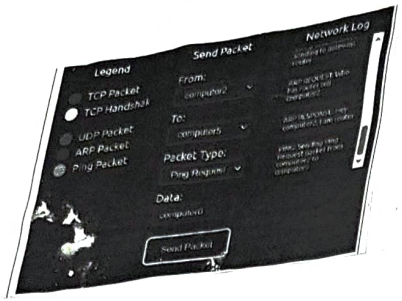
Nmap Advanced port scans

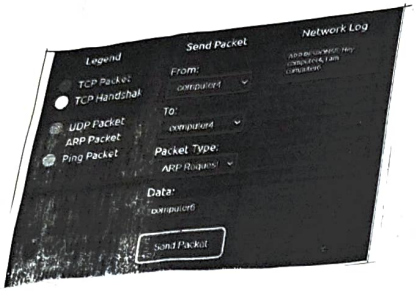
Nmap port port scans

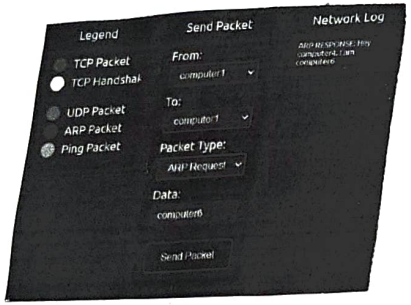



computer1
computer2
switch1
computer3
router
computer4
switch2
computer5
computer6


Legend
TCP Packet
TCP Handshak
UDP Packet
ARP Packet
Ping Packet
Send Packet
From:
computer1
To:
computer3
Packet Type:
Ping Request
Data:
computer6
Send Packet
Network Log







Result:

Three lives hosts are successful identified using NMAAP ARP scanning

b. ICMP Address mask. (-PM)

$ nmap -PM -SN 10.200.6.1024

Nmap done: 256 IP address (0 hosts up)
scanned in 52.14 seconds.

Tcp & UDP ping discovery.

(i) TCP SYN Ping

$ nmap -PS -SN 10.200.6.0/24

nmap scan report for 10.200.6.50
Host is up (0.20s latency)
Nmap done: 256 IP address

(ii) TCP ACK Ping (-PA)

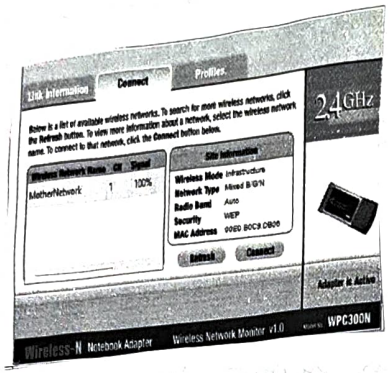
$ sudo nmap -PA -SA 10.200.6.0/24

Nmap scan report for 10.200.6.50
Host is up (0.020s Latency)
Nmap scan - 256 IP address

(iii) UDP Ping (-PU)

$ sudo nmap -PU - SN 10.200.6.0/24

Nmap scan report for 10.200.6.0.124
Nmap scan report for 10.200.6.51
Host is up (0.205 latency)
Nmap scan report for 10.200.6.250



Result:

Thus a secure wireless LAN was successfully configured using CISCO packet Tracer and all three PCs were connected to the wireless router.

Exp : 09

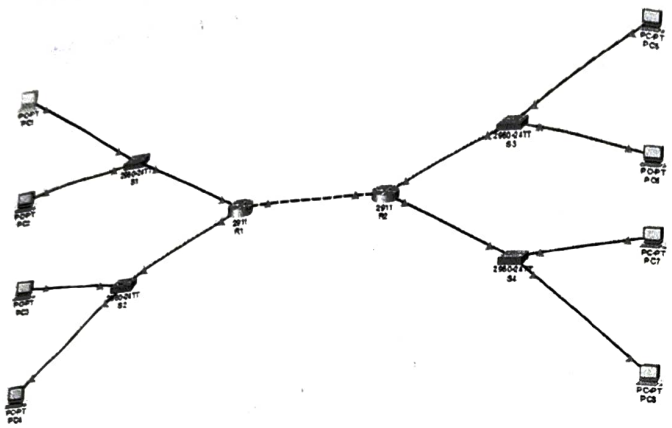
Implementation of Subnetting using the CISCO packet tracer

Aim:

To implement classless IP subnetting and configure communication between different subnets using cisco packet tracer.

Steps:

1. Subnetting is the process of dividing a large IP network into smaller networks called subnets.
2. classless subnetting was variable-length subnet mask instead of relaying only on the default class based masks.
3. In this experiment, the network 192.16.81.124 is dividing using the /24 subnetmask.
4. The subnet mask for /24 is 255.255.224
5. A /24 subnet provides 32 total address and 30 usable host addresses.
6. Subnetting helps in efficient IP address.
7. Different subnets can communicate through a router



Result:

Thus a classless IP subnetting was successfully implemented in cisco packet tracer and connectivity between the different subnets was verified using the ping command.

Exp 10:

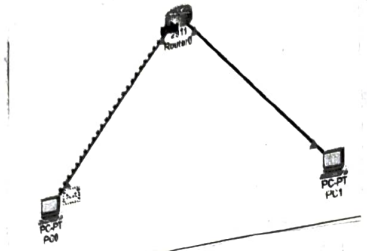
Inter networking with Routers in cisco packet tracer

AIM:

To design and configure a simple networking using a router and establish communication between two different networks

PROCEDURE:

1. Open cisco packet tracer and place one router and two PCs in the workspace.
2. connect PC0 to R1 using ethernet 0/0 and PC1 to R1 gigabit Ethernet 0/1 using copper-Straight through cables.
3. Configure R1 G0/0 with:
   IP : 192.168.10.1
   Subnet mask : 255.255.255.0
4. Configure R1 G0/1 with
   IP : 192.168.20.1
   Subnet mask : 255.255.255.0
5. Enable both router interfaces using no shutdown command.
6. Configure PC0 with:
   IP: 192.168.10.2
   subnet mask : 255.255.255.0
   Gateway : 192.168.10.0
7. IP : 192.168.20.2
   subnet mask : 255.255.255.0




```
exit
Router(config)#end
Router#
%SYS-6-CONFIG_I: Configured from console by console
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.10.1    YES manual up                    up
GigabitEthernet0/1     192.168.20.1    YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
Router#


Router con0 is now available
```

Result :

Thus a simple internetwork was successfully configured using a router and communication between 10no and PC1 was successfully verified.

(b) DESIGN AND CONFIGURE AN INTERWORK USING WIRELESS ROUTER, DHCP SERVER AND INTERNET CLOUD

AIM:

To design and configure an International using wireless router, DHCP server and internet cloud in cisco packet tracer.

PROCEDURE:

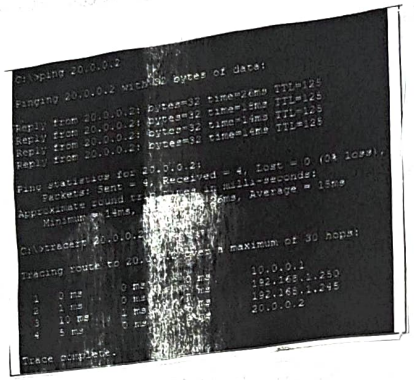
1. create the topography using a pc, laptop, wireless router, cable modem, internet cloud & cisco.com server.
2. connect the devices using appropriate cable
3. configure the laptop to wireless router using the SSID home network.
4. configure the laptop to connect to the wireless network
5. configure the internet cloud and cable modem connections.
6. configure the internet cloud and cable modem connections.
7. Test connectivity using ping 208.67.220.220 and ping cisco.com.

TOPOLOGY:

```
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```



RESULT:

Thus the internetwork was successfully configuration using a switch, router and ethernet cables and connectivity was verified using the ping command

Exp 11:

a. STATIC ROUTING CONFIGURATION USING CISCO PACKET TRACER

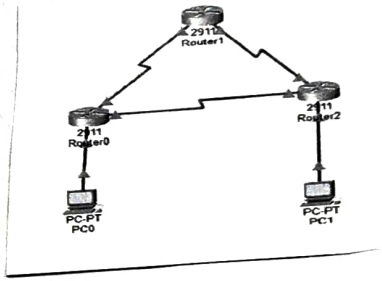
AIM:

To simulate static routing configuration using cisco packet tracer and verify the main and backup routes

THEORY:

Static routing is a routing method in which routes are manually configured in the routers routing table. A static route specifies the destination network, subnet mask, and next hop address. Administrative distance (AD) can be used to select a preffered route and configure a backup route. A route with a lower AD is preffered over a route with a higher AD.

TOPOLOGY:



IP Addressing Table

Device	Interface	
PC0	Fast Ethernet 0	10.0.0.0
Router 0	G0/0	10.0.0.1
Router 0	S0/3/0	20.0.0.1
Router 1	S0/3/0	20.0.0.2
Router 1	S0/3/1	50.0.0.2
Router 2	S0/3/0	40.0.0.2
Router 0	S0/3/1	30.0.0.1
Router 2	S0/3/1	20.0.0.1
Router 2	G0/0	30.0.0.10
PC1	Fast Ethernet 0	

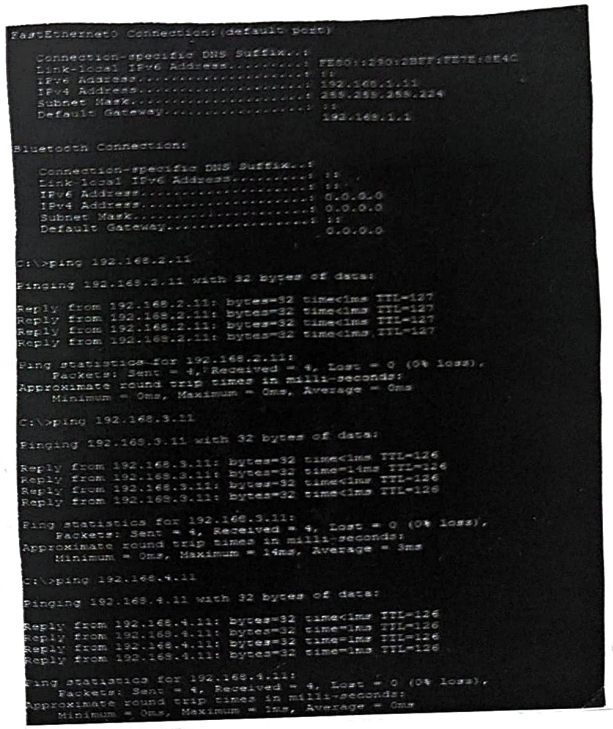
Subnet masks:

/8 - 255.0.0.0

/32 - 255.255.255.255

PROCEDURE:-

1. Create the network topology using three routers and two PCs in Cisco packet tracer.
2. Configure the IP address of the PCs and router interfaces.
3. Enable the router interfaces using the No shut down command
4. Configure static routing in router 0, Router 1 and Router 2.
5. Configure preffered and backup routes using different administrative distance value
6. Verify the router using show IP route static
7. Test connectivity using ping
8. Use tracert to verify the path followed by packets.
9. Disconnect the main route between Router
10. Verify that the backup route is automatical Routing:



Result: Thus static routing was successfully simulated using CISCO PACKET tracer. The main and backup static routes were configured and verified.

6) RIP VS/NO

Aim: To implement dynamic routing using RIP in cisco packet tracer and verify automatic route selection and rerouting.

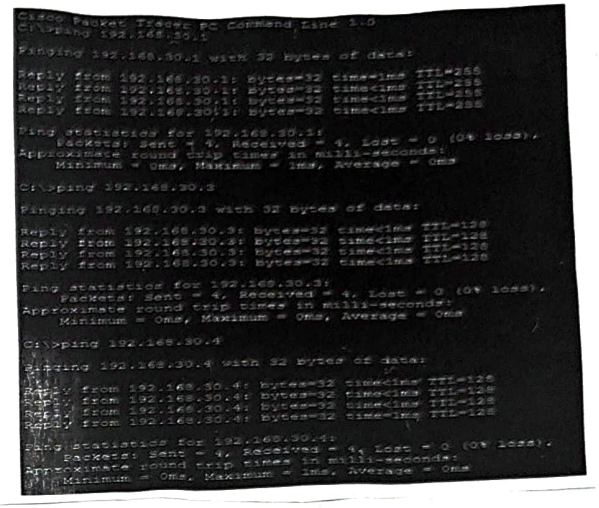
Theory:

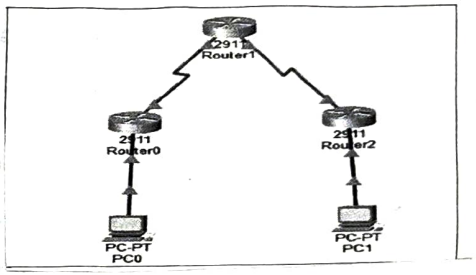
RIP (Routing Information protocol) is a distance vector protocol that uses hop count to select route. It automatically updates routing information when the network topology changes.

Procedure:

1. Create a network with 3 routers and 2 PCs
2. Configured IP addresses on PCs and router interfaces
3. Enabled all required router interfaces using no shutdown.
4. Configured RIP using the router and added the connected networks.
5. Tested connectivity between PC0 and PC1 using ping.
6. Used Tracert to verify the route selected by RIP
7. Disconnected the direct Router1 - Router2 link.
8. Verified that RIP automatically selected the alternate path through router 1

Topology:





Result:

Thus RIP dynamic routing was successfully implemented in cisco packet tracer and automatic route selection and rerouting were verified

Exp12: End to End communication

(a) implement echo client server using TCP/IP sockets

TCP:

```
def start_server
server = socket.socket(socket.AF_INET,
Socket.SOCK_STREAM)
server.bind('127.0.0.1', 5000)
server.listen()
conn, addr = server.accept()
print(f"[SERVER] connected by {addr}")
while True:
    data = conn.recv(1024).decode()
    if not data or data.lower()=='quit'
        print("[SERVER] connection closed")
client.")        break
    print(f"[SERVER] Recieved: {data}")
    conn.send(f"Server recieved: {data}
encode())
conn.close()
server.close()
```

```
# client
def start_client():
    client = socket.socket(socket.AF_INET, socket.
SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")
    while True:
        msg = input("Enter message (or 'quit' to
exit):")
        client.send(msg.encode())
        if msg.lower()=='quit':
            break
        response = client.recv(1024).decode
        print(f"[SERVER RESPONSE] {response}")
    client.close()

#main
if __name__=="__main__":
    # run server in a background thread
    server_thread = threading.Thread(target=
start_server, daemon=True)
    server_thread.start()
    #run client in main thread
    start_client()
```

(b) Implement a chat program using socket programming:

```
import socket
import threading

# server
def udp_server():
    s = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recv from(1024)
        msg = data.encode()
        if msg.lower() == 'quit':
        print(f"[SERVER] client {addr} ended the
connection.")
            break
        print(f"[SERVER] Received from {addr}:
{msg}")
        s.sendto(data, addr)

    s.close()
    print("[SERVER] Stopped!")
```

```
# client
def udp_client():
    c = socket.socket(socket.AF_INET, socket.
SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or 'quit' to
stop):")
        c.sendto(msg.encode(), server)
        if msg.lower() == "quit":
            print("[CLIENT] Closing connection...")
            break
        data, _ = c.recv from(1024)
        print("Echo from server:", data.decode())
    c.close()
    print("[CLIENT] Stopped!")

# main
if __name__ == "__main__":
    threading.Thread(target=udp_server)
    daemon=True).start(), udp_client()
```

14. write a code using RAW sockets to implement packet sniffing

```
from scapy.all import sniff
from scapy.layers.inet
import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
# determine the protocol
        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "Unknown protocol"
# print packet details
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("-" * 50)
# capture packets on the default network interface
sniff(iface="Wi-Fi", prn=packet_callback, filter="ip", store=0)
```

13. Implement your own ping problem.

```
import os
import platform
import subprocess

def ping(host="google.com", count=4):
    param = "-n" if platform.system().lower()
            =="windows" else "-c"
    command = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(command,
universal_newlines=True)
        print(output)
    except Exception as e:
        print("Ping failed: {e}")
if __name__ == "__main__":
    ping("8.8.8.8")
```

output:

```
Pinging 8.8.8.8 with 32 bytes of data
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8 bytes=32 time=7ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent=4, Received=4, Lost=0 (0% loss)
Approximate round trip times in milli-seconds:
    minimum=5ms, maximum=9ms, Average=7ms.
```